Flight delay insurance

Programmable benefits. Explicit oracle trust.

- CDS528 · Group project
- Mock assets only · Local prototype
- Report and code: https://github.com/DarkfireDK/Blockchain-Group-project

A precise problem

Travellers need predictable rules; LPs need visible exposure.

- Traveller: who receives the benefit, and when?
- LP: how much capital is reserved, and what is my share worth?
- Smart contracts enforce custody; a reporter still supplies the outcome.

Six contracts, one workflow

On-chain value and permissions; off-chain interaction.

CAPITAL

LP → InsurancePool

POLICY

Traveller → Insurance → NFT

OUTCOME

Reporter → Oracle → settlement

MockUSDC + InsurancePool → test asset, capital and shares

Insurance + PolicyNFT → terms, settlement and beneficiary

Oracle + PolicyMarket → reports and pre-departure sales

A policy has two different clocks

Time changes eligibility. A transaction finalises settlement.

- Before departure → market sale allowed
- Departure through end + 24 h → qualifying report allowed
- Strictly after end + 24 h → refund may be triggered

LP shares are not dollars

Net asset value and available liquidity answer different questions.

- Net assets = balance − refundable reserve − pending risk fee
- Available liquidity = balance − coverage reserve − refundable reserve
- New shares ≈ deposit × total shares ÷ net assets

Why 3,941.7476 shares can be worth 2,030

A reconciled local example, not a return forecast.

2,000
Initial capital

1,030
After settlements

2,030
After new deposit

2,000 deposit → 2,000 original shares

Two refunds + two claims → original assets fall to 1,030

Another 1,000 → 1,941.747571 new shares

Premium income does not guarantee profit

Fixed classroom pricing exposes a real economic limitation.

- 500 coverage → 10 premium → 5 refundable
- No claim: pool gains 5 · Claim: pool loses 490 net
- Simplified expected change = 5 − 495q

Security is layered

Permission checks, accounting invariants and atomic execution.

- Oracle and threshold snapshotted for every policy
- Roles + SafeERC20 + reentrancy guards
- Live ownership + pre-departure check + buyer maxPrice

Evidence, with boundaries

20 contract tests · 22 frontend logic tests · successful build

- Claims, refunds, permissions, reserves and market behaviour
- Boundary seconds, snapshot terms, donation and callback attacks
- Slither: 101 detectors; 11 retained findings

Demonstrate the complete lifecycle

Use two policies: one claim path and one refund path.

- LP approval + deposit → traveller approval + purchase
- Query policy → authorised report → current holder receives claim
- Separate policy → grace period ends → expiry refund

Deployment status is part of the result

Local implementation complete; testnet evidence still pending.

- Verified locally: deployment, tests and build
- Not yet evidenced: Sepolia + explorer source verification
- Before submission: public repo, updated code and successful CI

What we learned

Custody rules can be automated; external truth remains a dependency.

- Value: inspectable terms, ownership and reserve accounting
- Limits: manual oracle, teaching tariff and unfinished testnet proof
- Next: live-data trust model, calibrated pricing and browser acceptance

References & review checklist

Appendix · not included in the 12-minute speaking budget

- ERC-20 (2015) · ERC-721 (2018) · OpenZeppelin Contracts
- Hardhat 3 documentation · Crytic Slither
- Upstream: GtiHuberZZ/insurance-dapp · AI: ChatGPT/Codex